Bursa Uludağ University
Faculty of Engineering
Computer Engineering Department

**Wireless Network Protocols**

# SDN Data Plane and OpenFlow Flow Tables

Prepared by:
Sabir Süleymanlı
Student ID: 502531019

Date: 27/03/2026

# SDN Data Plane and OpenFlow Flow Tables

## 1. Instructions

**Task 1**: Building the Topology (Mininet Python API) Write a brief Python script (controller_topo.py) using the Mininet API to create a simple topology.

• 1 OpenFlow Switch (s1).

• 3 Hosts (h1, h2, h3) connected to s1.

• Important: Configure the network to connect to a Remote Controller (which will be your Ryu controller running on the same machine).

**Task 2**: Starting the Control Plane Instead of writing a controller from scratch, we will use a robust, pre-built application provided by Ryu.

1. Open a new terminal and start the Ryu L2 Learning Switch application using the following command: `ryu-manager ryu.app.simple_switch_13`

2. Run your Mininet topology script. You should see the switch connect to the controller in the Ryu terminal.

3. In the Mininet CLI, run `pingall`. Unlike a controller-less switch, this will succeed.

**Task 3**: Dynamic Flow Analysis The controller dynamically learns MAC addresses and pushes flow rules to the switch. Let's observe this:

1. In the Mininet CLI, clear all dynamically learned flows by running: `sh ovs-ofctl del-flows s1`

2. Open a terminal and run `sh ovs-ofctl dump-flows s1`. The table should be mostly empty (except for a default miss- rule).

3. In Mininet, `run h1 ping-c1 h2`.

4. Run `sh ovs-ofctl dump-flows s1` again.

**Task 4**: Overriding the Controller (Traffic Engineering) Currently, the Ryu controller allows all hosts to talk to each other. Your task is to implement a strict security policy. Isolate h3 from the network.

1. Keep the Ryu controller running.

2.Using the Mininet CLI, inject a manual, high-priority OpenFlow rule using ovs-ofctl add-flow to drop any packets going to or coming from h3's IP or MAC address.

3. Verify your policy by running `pingall`. h1 and h2 should communicate, but h3 should be completely unreachable.

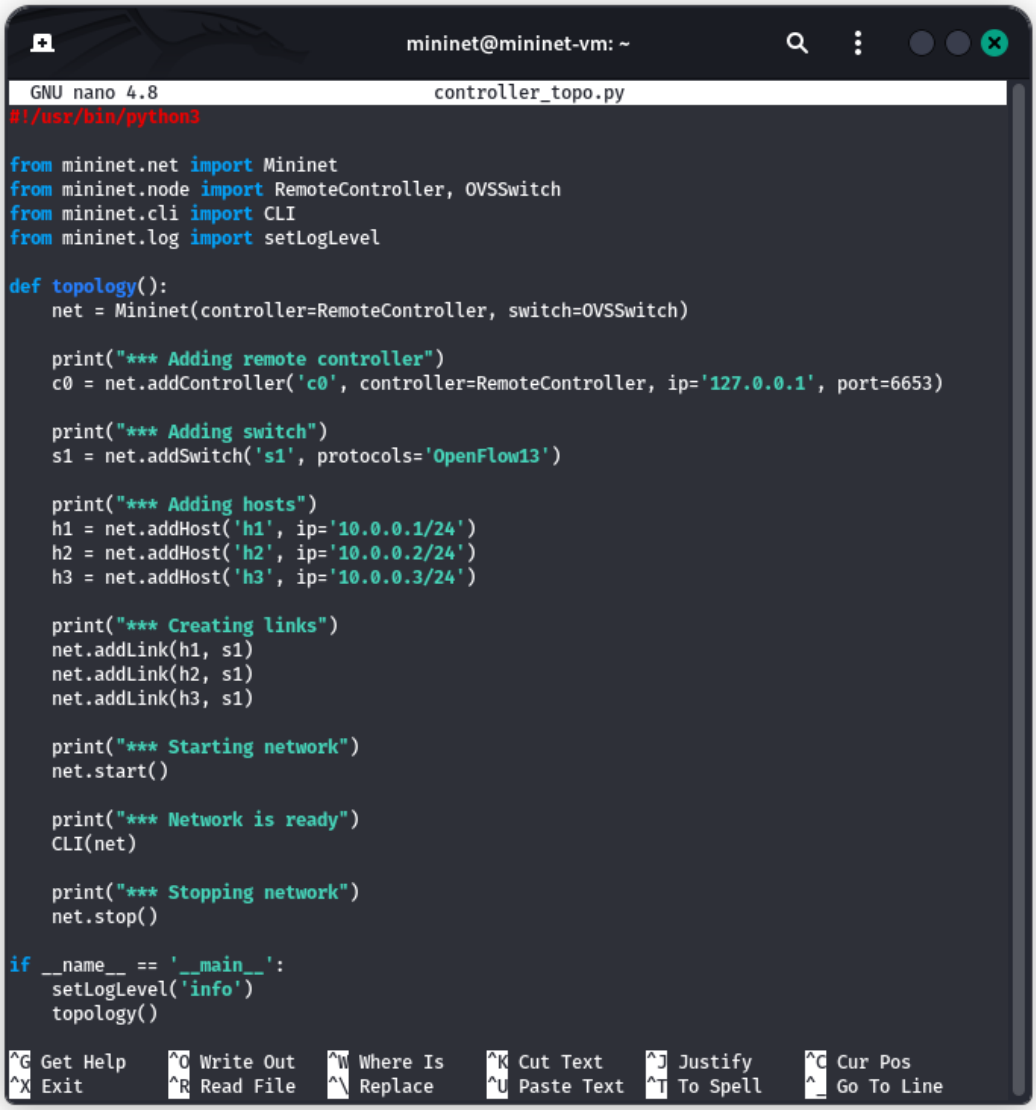
---

## 2. Task 1

In this task, I first created the `controller_topo.py` file using the Mininet Python API and defined one OpenFlow switch (`s1`) and three hosts (`h1`, `h2`, `h3`) within the file. I connected these hosts to the `s1` switch and configured the necessary setup to enable the network to communicate with a remote controller.

On the switch side, I preferred to use the OpenFlow 1.3 protocol in order to ensure compatibility with the Ryu controller.

**controller_topo.py** source code**:**




```python
#!/usr/bin/python3

from mininet.net import Mininet
from mininet.node import RemoteController, OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel

def topology():
    net = Mininet(controller=RemoteController, switch=OVSSwitch)

    print("*** Adding remote controller")
    c0 = net.addController('c0', controller=RemoteController, ip='127.0.0.1', port=6653)

    print("*** Adding switch")
    s1 = net.addSwitch('s1', protocols='OpenFlow13')

    print("*** Adding hosts")
    h1 = net.addHost('h1', ip='10.0.0.1/24')
    h2 = net.addHost('h2', ip='10.0.0.2/24')
    h3 = net.addHost('h3', ip='10.0.0.3/24')

    print("*** Creating links")
    net.addLink(h1, s1)
    net.addLink(h2, s1)
    net.addLink(h3, s1)

    print("*** Starting network")
    net.start()

    print("*** Network is ready")
    CLI(net)

    print("*** Stopping network")
    net.stop()

if __name__ == '__main__':
    setLogLevel('info')
    topology()
```

Output:

```
mininet@mininet-vm: ~
mininet@mininet-vm:~$ sudo python3 controller_topo.py

*** Adding remote controller
Unable to contact the remote controller at 127.0.0.1:6653
*** Adding switch
*** Adding hosts
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Network is ready
*** Starting CLI:
mininet>
```

---

## 3. Task 2

In this step, I activated the control plane. I opened a new terminal and started the Ryu controller using the command `python3 -m ryu.cmd.manager ryu.app.simple_switch_13`. The `simple_switch_13` application learns MAC addresses by analyzing the packets traversing the network and dynamically installs appropriate forwarding rules on the switch.

```
mininet@mininet-vm: ~
mininet@mininet-vm: ~    mininet@mininet-vm: ~
mininet@mininet-vm:~$ python3 -m ryu.cmd.manager ryu.app.simple_switch_13
3 RLock(s) were not greened, to fix this error make sure you run eventlet.monkey_patch() be
fore importing any other modules.
loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
```

From this output, it can be observed that the switch successfully connected to the controller. Afterwards, I ran the `pingall` command in the Mininet CLI on the first terminal to test the communication between the hosts.

```
mininet@mininet-vm: ~
mininet@mininet-vm: ~    mininet@mininet-vm: ~
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Network is ready
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet>
```

From this, it can be understood that the switch forwards traffic according to the rules received from the controller. This highlights the critical role of the control plane in the SDN architecture.

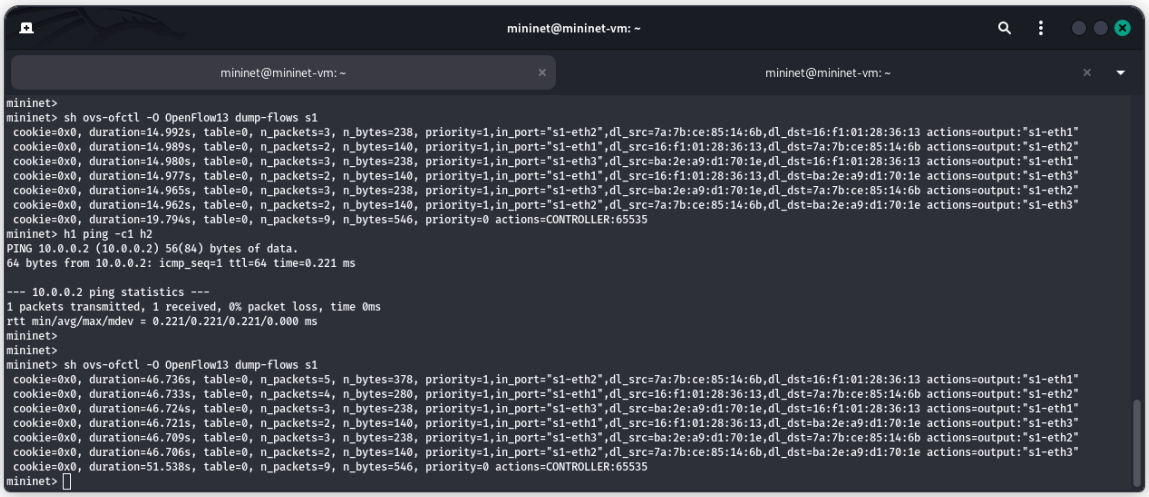
---

## 4. Task 3

In this step, I examined the flow table structure on the switch and observed the dynamic behavior of the controller. First, I inspected the existing flow rules on the switch. Then, I generated traffic by sending a ping from host `h1` to host `h2` using the `h1 ping -c1 h2` command:

```
mininet@mininet-vm: ~
mininet@mininet-vm: ~    mininet@mininet-vm: ~
mininet>
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=14.992s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth2",dl_src=7a:7b:ce:85:14:6b,dl_dst=16:f1:01:28:36:13 actions=output:"s1-eth1"
 cookie=0x0, duration=14.989s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth1",dl_src=16:f1:01:28:36:13,dl_dst=7a:7b:ce:85:14:6b actions=output:"s1-eth2"
 cookie=0x0, duration=14.980s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth3",dl_src=ba:2e:a9:d1:70:1e,dl_dst=16:f1:01:28:36:13 actions=output:"s1-eth1"
 cookie=0x0, duration=14.977s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth1",dl_src=16:f1:01:28:36:13,dl_dst=ba:2e:a9:d1:70:1e actions=output:"s1-eth3"
 cookie=0x0, duration=14.965s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth3",dl_src=ba:2e:a9:d1:70:1e,dl_dst=7a:7b:ce:85:14:6b actions=output:"s1-eth2"
 cookie=0x0, duration=14.962s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth2",dl_src=7a:7b:ce:85:14:6b,dl_dst=ba:2e:a9:d1:70:1e actions=output:"s1-eth3"
 cookie=0x0, duration=19.794s, table=0, n_packets=9, n_bytes=546, priority=0 actions=CONTROLLER:65535
mininet> h1 ping -c1 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.221 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.221/0.221/0.221/0.000 ms
mininet>
```

Afterwards, when I checked the flow table again using the same command, I observed that the controller had created dynamic forwarding rules that determine which ports should be used for specific MAC addresses:



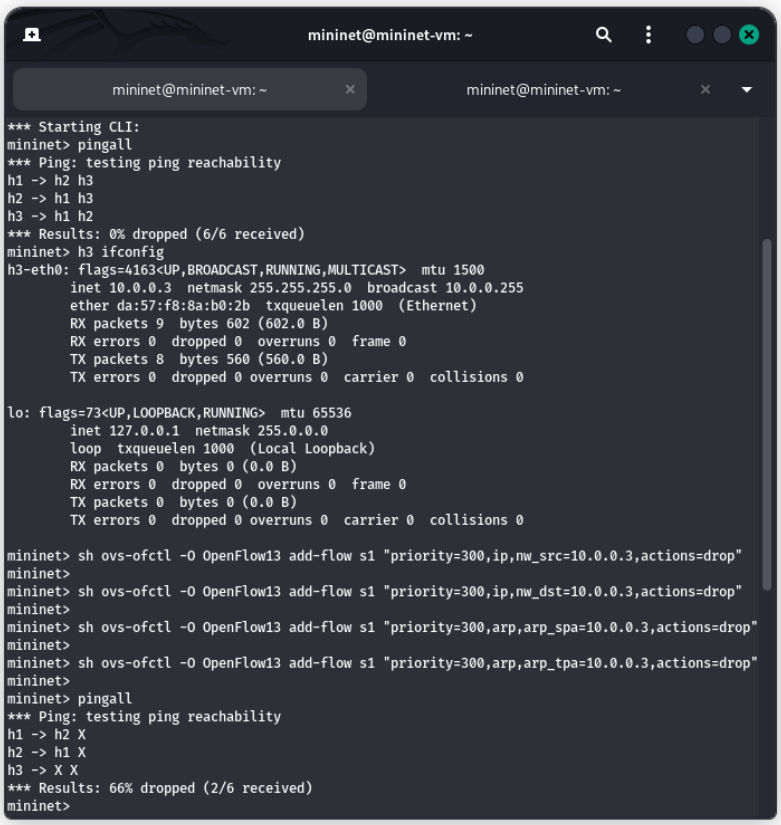
```
mininet@mininet-vm: ~
mininet@mininet-vm: ~    mininet@mininet-vm: ~
mininet>
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=14.992s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth2",dl_src=7a:7b:ce:85:14:6b,dl_dst=16:f1:01:28:36:13 actions=output:"s1-eth1"
 cookie=0x0, duration=14.989s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth1",dl_src=16:f1:01:28:36:13,dl_dst=7a:7b:ce:85:14:6b actions=output:"s1-eth2"
 cookie=0x0, duration=14.980s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth3",dl_src=ba:2e:a9:d1:70:1e,dl_dst=16:f1:01:28:36:13 actions=output:"s1-eth1"
 cookie=0x0, duration=14.977s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth1",dl_src=16:f1:01:28:36:13,dl_dst=ba:2e:a9:d1:70:1e actions=output:"s1-eth3"
 cookie=0x0, duration=14.965s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth3",dl_src=ba:2e:a9:d1:70:1e,dl_dst=7a:7b:ce:85:14:6b actions=output:"s1-eth2"
 cookie=0x0, duration=14.962s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth2",dl_src=7a:7b:ce:85:14:6b,dl_dst=ba:2e:a9:d1:70:1e actions=output:"s1-eth3"
 cookie=0x0, duration=19.794s, table=0, n_packets=9, n_bytes=546, priority=0 actions=CONTROLLER:65535
mininet> h1 ping -c1 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.221 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.221/0.221/0.221/0.000 ms
mininet>
mininet>
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=46.736s, table=0, n_packets=5, n_bytes=378, priority=1,in_port="s1-eth2",dl_src=7a:7b:ce:85:14:6b,dl_dst=16:f1:01:28:36:13 actions=output:"s1-eth1"
 cookie=0x0, duration=46.733s, table=0, n_packets=4, n_bytes=280, priority=1,in_port="s1-eth1",dl_src=16:f1:01:28:36:13,dl_dst=7a:7b:ce:85:14:6b actions=output:"s1-eth2"
 cookie=0x0, duration=46.724s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth3",dl_src=ba:2e:a9:d1:70:1e,dl_dst=16:f1:01:28:36:13 actions=output:"s1-eth1"
 cookie=0x0, duration=46.721s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth1",dl_src=16:f1:01:28:36:13,dl_dst=ba:2e:a9:d1:70:1e actions=output:"s1-eth3"
 cookie=0x0, duration=46.709s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth3",dl_src=ba:2e:a9:d1:70:1e,dl_dst=7a:7b:ce:85:14:6b actions=output:"s1-eth2"
 cookie=0x0, duration=46.706s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth2",dl_src=7a:7b:ce:85:14:6b,dl_dst=ba:2e:a9:d1:70:1e actions=output:"s1-eth3"
 cookie=0x0, duration=51.538s, table=0, n_packets=9, n_bytes=546, priority=0 actions=CONTROLLER:65535
mininet>
```

Based on the outputs, it can be seen that the switch acts as a passive component in the SDN architecture, while the actual decision-making process is managed by the controller.

---

## 5. Task 4

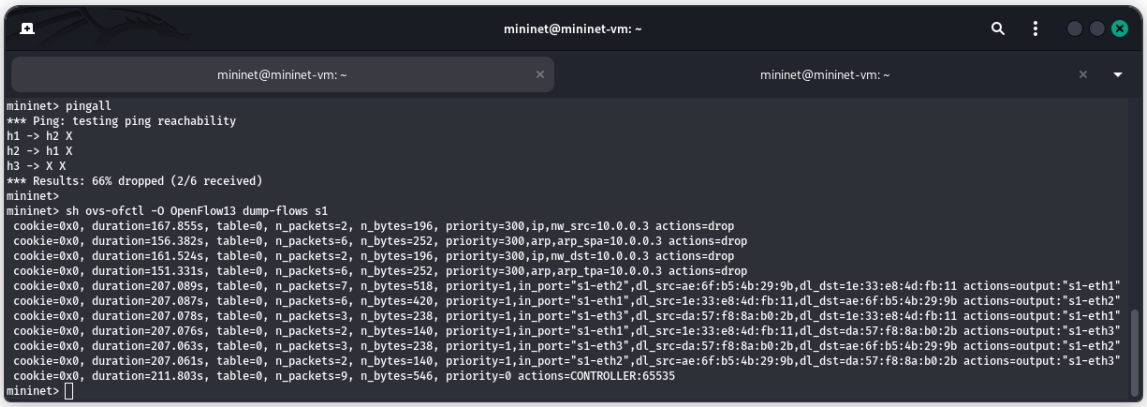
In this step, I manually intervened in the behavior of the Ryu controller in order to isolate `h3`. Under normal conditions, the Ryu controller allows all hosts to communicate with each other. I added high-priority OpenFlow rules to the switch via the Mininet CLI. With these rules, I ensured that both IP and ARP traffic originating from and destined to host `h3` were dropped:



```
mininet@mininet-vm: ~
mininet@mininet-vm: ~    mininet@mininet-vm: ~
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet> h3 ifconfig
h3-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.3  netmask 255.255.255.0  broadcast 10.0.0.255
        ether da:57:f8:8a:b0:2b  txqueuelen 1000  (Ethernet)
        RX packets 9  bytes 602 (602.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 560 (560.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

mininet> sh ovs-ofctl -O OpenFlow13 add-flow s1 "priority=300,ip,nw_src=10.0.0.3,actions=drop"
mininet>
mininet> sh ovs-ofctl -O OpenFlow13 add-flow s1 "priority=300,ip,nw_dst=10.0.0.3,actions=drop"
mininet>
mininet> sh ovs-ofctl -O OpenFlow13 add-flow s1 "priority=300,arp,arp_spa=10.0.0.3,actions=drop"
mininet>
mininet> sh ovs-ofctl -O OpenFlow13 add-flow s1 "priority=300,arp,arp_tpa=10.0.0.3,actions=drop"
mininet>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 X
h2 -> h1 X
h3 -> X X
*** Results: 66% dropped (2/6 received)
mininet>
```

Here, by using a `priority = 300`, I ensured that these rules take precedence over the ones installed by the controller. As a result, even though the controller remained active, the security policy I defined was successfully enforced.

Finally, when examining the flow table, it can be observed that the drop rules I added appear with high priority. This clearly demonstrates that these rules override the forwarding rules created by the controller.




You can refer to the relevant GitHub repository to review the source code and other details:

https://github.com/sabir-suleyman/sdn-mininet-ryu